

LECTURE 05

Navigation & Routing

Moving between screens in your Flutter app

Instructor · Muhammad Sabah

Today's plan

01

Navigator basics

push & pop — go forward, come back

02

Passing data

Send data forward, return data back

03

Named routes

Define routes once, reuse anywhere

04

Practice

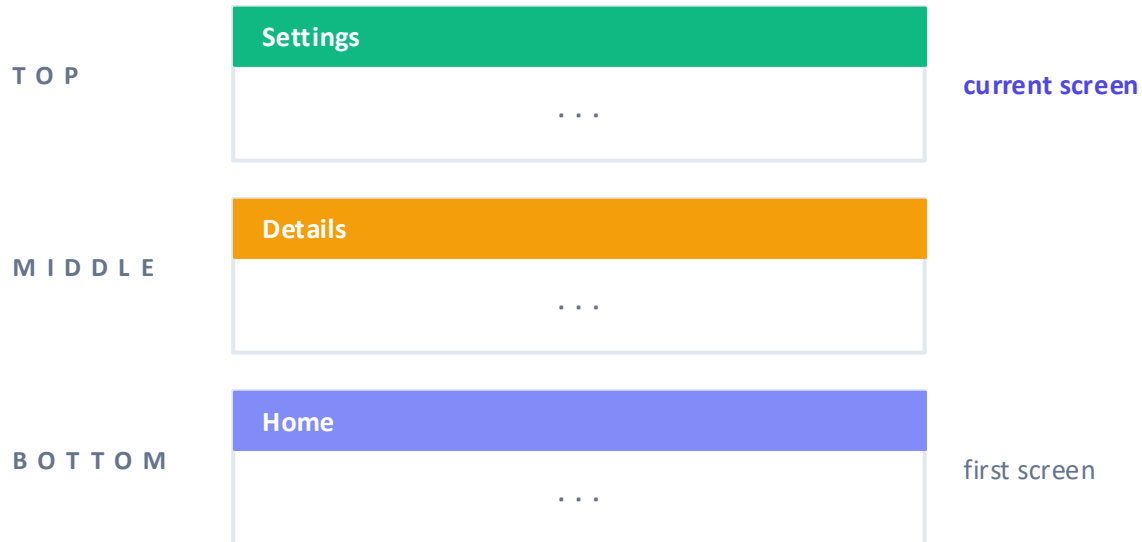
Two hands-on exercises in class

THE BIG IDEA

Navigator is a stack of screens

Every Flutter app keeps a stack of pages. push adds one on top, pop removes the top one.

THE STACK



TWO OPERATIONS

push

Add a new screen on top of the stack — go forward.

pop

Remove the top screen — go back to the previous one.

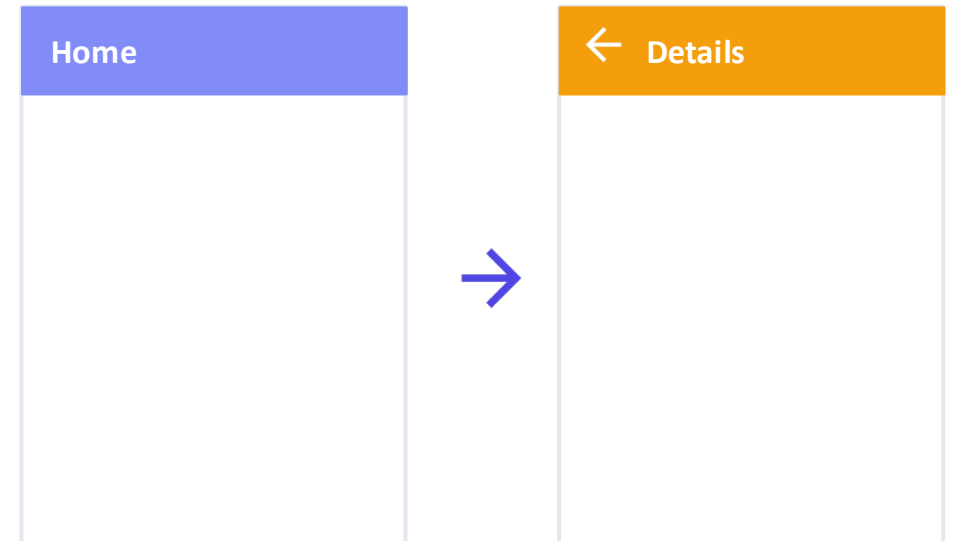
Navigator.push — go to a new screen

Use `Navigator.push` to open a new screen on top of the current one.

ON BUTTON PRESS

```
ElevatedButton(  
  onPressed: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(  
        builder: (context) => DetailsScreen(),  
      ),  
    );  
  },  
  child: Text('Open Details'),  
)
```

WHAT HAPPENS



MaterialPageRoute gives you a smooth slide animation for free.

Navigator.pop — come back

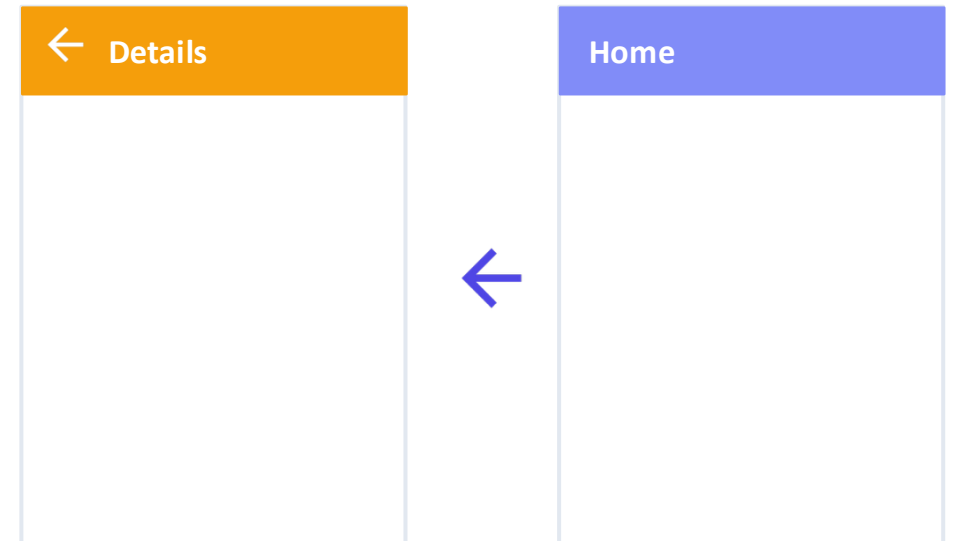
Use `Navigator.pop` to close the current screen and return to the previous one.

ON BUTTON PRESS

```
ElevatedButton(  
  onPressed: () {  
    Navigator.pop(context);  
  },  
  child: Text('Go Back'),  
)
```

The back arrow in the AppBar already calls `Navigator.pop` automatically.

WHAT HAPPENS



Passing data to the next screen

Pass data through the new screen's constructor.

SENDING (HOME SCREEN)

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (context) =>  
      ProfileScreen(name: 'Hama'),  
  ),  
);
```

RECEIVING (PROFILE SCREEN)

```
class ProfileScreen extends StatelessWidget {  
  final String name;  
  
  ProfileScreen({required this.name});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Text('Hello, $name!'),  
    );  
  }  
}
```

Returning data when popping

Navigator.push returns a Future — await it to get the value from pop.

SECOND SCREEN — POP WITH VALUE

```
ElevatedButton(  
  onPressed: () {  
    Navigator.pop(context, 'Yes');  
  },  
  child: Text('Confirm'),  
)
```

FIRST SCREEN — AWAIT THE RESULT

```
onPressed: () async {  
  final result = await Navigator.push(  
    context,  
    MaterialPageRoute(  
      builder: (context) => ConfirmScreen(),  
    ),  
  );  
  
  print('User said: $result');  
}
```

Defining named routes

Give each screen a name in MaterialApp — then refer to it by string.

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => HomeScreen(),  
    '/details': (context) => DetailsScreen(),  
    '/settings': (context) => SettingsScreen(),  
  },  
)
```

WHY USE THEM

Cleaner code

No long MaterialPageRoute everywhere

One place

All routes defined in one spot

Easier deep links

Routes match URL-like paths

Navigator.pushNamed

Push by name instead of building the route manually.

BEFORE — LONG

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (context) =>  
      DetailsScreen(),  
  ),  
);
```

AFTER — SHORT & CLEAN

```
Navigator.pushNamed(  
  context,  
  '/details',  
);  
  
// Works because '/details' is  
// defined in MaterialApp.routes.
```

Use Navigator.pop(context) the same way to go back — that part does not change.

Build two screens, push & pop

Make a Home screen with a button that opens a Details screen.

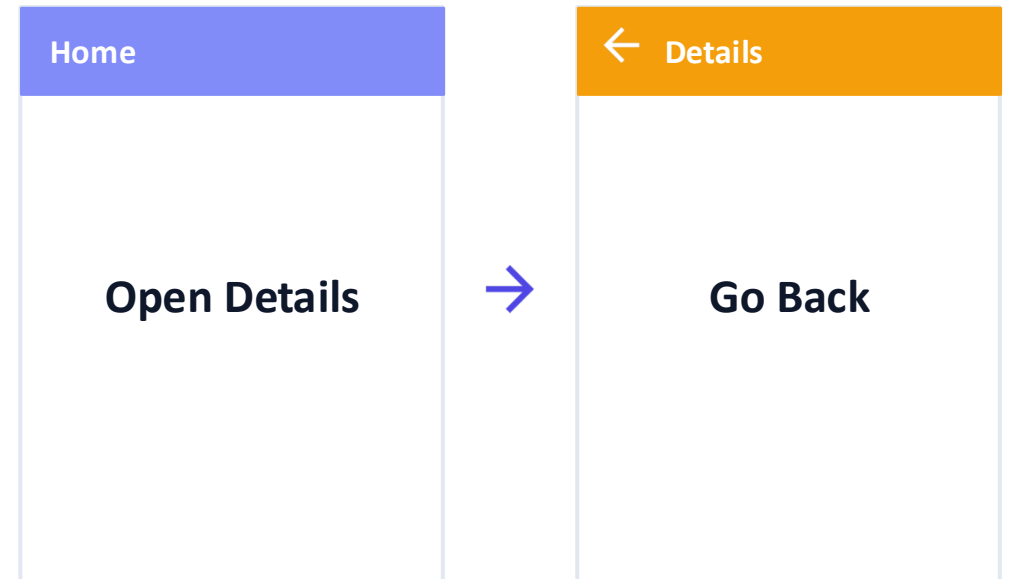
YOUR TASK

1. Create HomeScreen with an ElevatedButton
2. On press, push DetailsScreen
3. DetailsScreen shows a Text and a button
4. The button pops back to Home

HINT

```
Navigator.push(context,  
  MaterialPageRoute(builder: (_) => DetailsScreen()),  
);
```

EXPECTED FLOW



Pass a name to a Greeting screen

On Home, take input then push to a Greeting screen that says hello with the name.

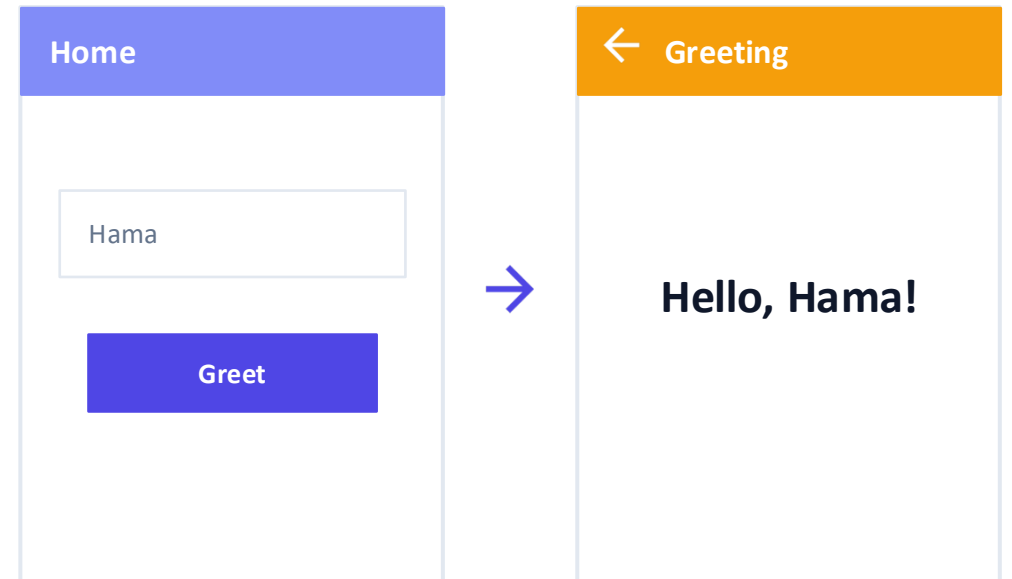
YOUR TASK

1. Add a TextField on HomeScreen
2. Add a button — read the field's value
3. Push GreetingScreen, passing the name
4. GreetingScreen displays: Hello, {name}!

HINT

```
final controller = TextEditingController();  
// then on press:  
GreetingScreen(name: controller.text)
```

EXPECTED FLOW



W R A P U P

What we covered today

Y O U N O W K N O W

- ✓ Navigator is a stack of screens
- ✓ Navigator.push to go forward
- ✓ Navigator.pop to come back
- ✓ Passing data through constructors
- ✓ Returning data with await + pop
- ✓ Defining routes in MaterialApp
- ✓ Navigator.pushNamed for clean code

T H A N K Y O U

Questions?

Muhammad Sabah · hama-sabah.dev